

```
1 using System;
2 using System.Linq;
3 using System.Net.Sockets;
4 using System.Threading;
5 using SharpDX.DirectInput;
6
7 class Program
8 {
9     static void Main()
10    {
11        // ===== PLC接続設定 =====
12        string plcIp = "192.168.3.250";
13        int plcPort = 5000;
14
15        TcpClient client = new TcpClient();
16        client.Connect(plcIp, plcPort);
17        NetworkStream stream = client.GetStream();
18
19        // ===== DirectInput初期化 =====
20        var directInput = new DirectInput();
21        var joystickGuid = Guid.Empty;
22
23        /*
24        foreach (var device in directInput.GetDevices(DeviceType.Joystick,
25            DeviceEnumerationFlags.AllDevices))
26        {
27            joystickGuid = device.InstanceGuid;
28        }
29
30        if (joystickGuid == Guid.Empty)
31        {
32            Console.WriteLine("ゲームパッドが見つかりません");
33            return;
34        }
35
36        var joystick = new Joystick(directInput, joystickGuid);
37        joystick.Properties.BufferSize = 128;
38        joystick.Acquire();
39        */
40        var devices = directInput.GetDevices(DeviceClass.GameControl,
41            DeviceEnumerationFlags.AttachedOnly);
42
43        if (devices.Count == 0)
44        {
45            Console.WriteLine("コントローラが見つかりません");
46            return;
47        }
48        // 最初のデバイスを使用
49        var joystick = new Joystick(directInput, devices[0].InstanceGuid);
50
51        joystick.Properties.BufferSize = 128;
52        joystick.Acquire();
53
54        Console.WriteLine("開始");
55
56        while (true)
57        {
58            joystick.Poll();
```

```
59         var state = joystick.GetCurrentState();
60
61         int y = state.Y; // 0~65535
62
63         byte value;
64
65         if (y > 32768 + 2000)
66             value = 0x01;
67         else if (y < 32768 - 2000)
68             value = 0x02;
69         else
70             value = 0x00;
71
72         byte[] message = CreateSimpWriteCommand(value);
73
74         // ===== 送信 =====
75         stream.Write(message, 0, message.Length);
76
77         // ===== 応答受信 (軽く) =====
78         byte[] buffer = new byte[256];
79         stream.Read(buffer, 0, buffer.Length);
80
81         Console.WriteLine($"Y={y} -> {value:X2}");
82
83         Thread.Sleep(100);
84     }
85 }
86
87 static byte[] CreateSimpWriteCommand(byte value)
88 {
89     byte[] command =
90     {
91         0x01, 0x14, // コマンド (1401H → リトルエンディアン)
92         0x00, 0x00, // サブコマンド
93
94         0x01, 0x00, 0x00, // 先頭デバイス番号 (000001H)
95         0xA8, // デバイスコード (Dレジスタ)
96
97         0x01, 0x00, // 点数 (1点)
98
99         value, 0x00 // 書き込みデータ (1ワード)
100     };
101
102     return Create3EFrame(command);
103 }
104
105 static byte[] Create3EFrame(byte[] command)
106 {
107     var msg = new System.Collections.Generic.List<byte>();
108
109     // サブヘッダ
110     msg.AddRange(new byte[] { 0x50, 0x00 });
111
112     // アクセス経路
113     msg.Add(0x00); // ネットワーク番号
114     msg.Add(0xFF); // PC番号
115     msg.AddRange(new byte[] { 0xFF, 0x03 }); // I/O番号
116     msg.Add(0x00); // 局番号
117
118     // データ長 (監視タイマ + 要求データ)
```

```
119         int length = 2 + command.Length;
120         msg.Add((byte)(length & 0xFF)); // 下位
121         msg.Add((byte)((length >> 8) & 0xFF)); // 上位
122
123         // 監視タイマ
124         msg.Add(0x00);
125         msg.Add(0x00);
126
127         // コマンド
128         msg.AddRange(command);
129
130         return msg.ToArray();
131     }
132 }
133
```